



Nome: _____ Data: _____ Nota: _____

Disciplina: Estrutura de Dados

Curso: CST-ADS

Professor: Marcelo Paravisi e Felipe Costa

Avaliação 2

Questões

1. Considere o vetor abaixo, aplique o algoritmo de ordenação por inserção. Ou seja, mostre o estado do vetor após cada iteração do algoritmo (cada inserção).

[9, 5, 1, 4, 3]

- **Vetor inicial:** [9, 5, 1, 4, 3]
- **Iteração 1: Vetor:** [5, 9, 1, 4, 3]
- **Iteração 2: Vetor:** [1, 5, 9, 4, 3]
- **Iteração 3: Vetor:** [1, 4, 5, 9, 3]
- **Iteração 4: Vetor:** [1, 3, 4, 5, 9]

2. Considere o vetor abaixo, aplique o algoritmo de ordenação por seleção. Ou seja, mostre a cada iteração o menor elemento encontrado e a troca realizada, bem como o vetor: [9, 5, 1, 4, 3].

Vetor inicial: [9, 5, 1, 4, 3]

Iteração 1:

Menor elemento encontrado: 1 (no índice 2).

Troca realizada: Troca-se o 9 (índice 0) pelo 1 (índice 2).

Vetor: [1, 5, 9, 4, 3]

Iteração 2:

Menor elemento encontrado: 3 (no índice 4).

Troca realizada: Troca-se o 5 (índice 1) pelo 3 (índice 4).

Vetor: [1, 3, 9, 4, 5]

Iteração 3:

Menor elemento encontrado: 4 (no índice 3).

Troca realizada: Troca-se o 9 (índice 2) pelo 4 (índice 3).

Vetor: [1, 3, 4, 9, 5]

Iteração 4:

Menor elemento encontrado: 5 (no índice 4).

Troca realizada: Troca-se o 9 (índice 3) pelo 5 (índice 4).

Vetor: [1, 3, 4, 5, 9]

(A última posição já fica garantida como o maior elemento).



3. Observe o vetor abaixo. Utilizando o QuickSort com o primeiro elemento como pivô, mostre como ficará o vetor após a primeira partição.

[7, 4, 9, 3, 10, 5, 1, 11]

Vetor inicial: [7, 4, 9, 3, 10, 5, 1, 11]

Pivô escolhido: 7 (o primeiro elemento).

Vetor após a primeira partição:

[4, 3, 5, 1, 7, 9, 10, 11]

4. Explique com suas palavras como funciona o algoritmo QuickSort. Em sua resposta, descreva o papel do pivô, o processo de partição e como ocorre a recursividade no algoritmo.

Ideia geral: O QuickSort é um algoritmo de ordenação que segue a estratégia de divisão e conquista. Ele escolhe um pivô, particiona o vetor em duas partes (elementos menores que o pivô e elementos maiores ou iguais) e então aplica recursivamente o mesmo processo em cada parte.

Papel do pivô: O pivô é o elemento usado como referência para dividir o vetor. Após a partição, o pivô fica na posição final correta no vetor ordenado, porque tudo à esquerda é menor e tudo à direita é maior (no caso de ordem crescente).

Processo de partição: No particionamento, percorremos o vetor, comparando cada elemento com o pivô. Quando encontramos elementos menores que o pivô, colocamos esses elementos em uma região à esquerda; os maiores ficam à direita. No final, colocamos o pivô entre essas duas regiões.

Recursividade: Depois de particionar, o problema é dividido em dois subproblemas menores: ordenar o subvetor da esquerda e o subvetor da direita. O QuickSort chama a si mesmo (recursivamente) para cada subvetor, até que os subvetores tenham tamanho 0 ou 1, o que significa que já estão ordenados.



5. Crie um programa que obtenha 10 números inteiros digitados pelo usuário e os armazene em um array. Implemente o algoritmo de ordenação Bubble Sort para ordenar os valores. Ao final, mostre o array ordenado.

```
#include <stdio.h>

int main() {
    int vet[10];
    int i, j, temp;

    /* leitura dos 10 números inteiros */
    for (i = 0; i < 10; i++) {
        printf("Digite o %do numero: ", i + 1);
        scanf("%d", &vet[i]);
    }

    /* algoritmo Bubble Sort (ordem crescente) */
    for (i = 0; i < 10 - 1; i++) {
        for (j = 0; j < 10 - i - 1; j++) {
            if (vet[j] > vet[j + 1]) {
                temp = vet[j];
                vet[j] = vet[j + 1];
                vet[j + 1] = temp;
            }
        }
    }

    /* exibindo o array ordenado */
    printf("Vetor ordenado: ");
    for (i = 0; i < 10; i++) {
        printf("%d ", vet[i]);
    }
    printf("\n");

    return 0;
}
```